NUS ORBITAL 2026 — APOLLO LEVEL

O(1) Exchange — Milestone 1

Ultra-Low Latency In-Memory Matching Engine & Live Trading Interface

DEVELOPED BY

Shrey Jain & Akshay

Apollo Candidates

NUS Computing

The Vision & Core Idea

Standard financial platforms are bottlenecked by disk-bound databases, introducing microsecond latencies that are fatal to high-frequency trades.

O(1) Exchange is a specialized, **hardware-optimized order book** engineered entirely in C++. By storing trade states in lock-free RAM blocks and using L3 cache alignment, matching operations execute in pure **sub-millisecond intervals**. Trade feeds are then streamed instantly via WebSocket to a dynamic React dashboard.

`C++20` `Boost.Beast` `React` `SQLite` `L3-Optimized`

System Architecture

The system isolates trading instruments to prevent concurrent resource blocking, creating a truly scalable multi-asset market.

React Trading UI
WebSocket Client | State Rendering

`ws://host:9001 (JSON messages)`

Boost.Beast WebSocket Server
Multi-Client | Detached Thread Worker Pool

`Synchronized In-Memory Dispatch`

O(1) In-Memory Matching Engine
Per-Instrument Book Mutex | 3-Level Bitmaps

`SQLite Atomic Ledger Updates`

SQLite Account Registry & Security
libsodium Argon2id | Transaction Batching

Bitmapped Fast Paths: Uses 3-level bit arrays (`bids_l1_`, `bids_l2_`, `bids_l3_`) for empty-level checks. Employs hardware-level intrinsics (`__builtin_clzll` and `__builtin_ctzll`) to locate the best bid or ask in absolute **O(1) time**, avoiding expensive map iterations.

Core Engine Ledger (Proof of Concept)

A high-fidelity visual representation of the real-time account ledger state, limit order book depth, and trade execution tape generated by our integrated matching engine.

Account Registry Ledgers

SHREY (BUYER)	AKSHAY (SELLER)
$16,450.00	$20,000.00
APL Available: 500 | Escrow Reserved: $33,550.00	APL Available: 50 | Escrow Reserved: 450 shares

In-Memory Order Book Depth (APL)

SIDE	PRICE	QUANTITY	COUNT
SELL	$102	150	1
SELL	$105	300	1
Spread: $4.00			
BUY	$98	100	1
BUY	$95	250	1

Real-Time Execution Tape

```
APL matched $99.00 150 shares Shrey (B) ↔ Akshay (S)
APL matched $100.00 100 shares Akshay (B) ↔ Shrey (S)
APL matched $101.50 50 shares Shrey (B) ↔ Akshay (S)
```

The Engineering Roadmap

1. **MILESTONE 1 (CURRENT)**
   Correctness & Hardening
   Completed safety integration: Secure session tracking (libsodium CSPRNG), SQL transactional persistence, and zero-leak escrow loops.
2. **MILESTONE 2 (JUNE)**
   Scale & Dynamic Listing
   Expose full REST API, support dynamically registered instruments via administrative dashboard, and stress-test the WebSocket layer.
3. **MILESTONE 3 (JULY)**
   Market Orders & Charts
   Add Stop-Loss & Market order executions. Integrate interactive price candle charts on React UI. Harden multi-threaded safety.

Transition to Milestone 1

While our Liftoff model demonstrated basic order flows, transitioning to Milestone 1 involved conducting a rigorous, exhaustive codebase audit. We identified and fortified critical architectural vulnerabilities.

Escrow Leaks on Cancel — CRITICAL

Symptom: Orders reserve funds/shares upfront. If an order was cancelled or partially filled, the unfilled remainder was never refunded, permanently locking assets in database escrow.

✓ Resolved: Linked Engine cancellation callbacks to an atomic SQLite escrow-release protocol.

Monotonic Collision (L3 Cache) — CRITICAL

Symptom: monotonically increasing order IDs wrapped around MAX_ORDERS, causing key collisions that silently overwrote and orphaned live resting orders.

✓ Resolved: Optimized pool sizes to fit within CPU L3 cache and mapped IDs via SPSC index + rolling epoch tracking.

DB Transaction Flickering — HIGH

Symptom: DB mutex was released inside loops during settlements. UI threads read ledger states mid-batch, causing visual wallet fluctuations.

✓ Resolved: Enforced exclusive lock acquisition across the entirety of SQLite batch transactions.

Broadcast Queue Bottlenecks — HIGH

Symptom: Network broadcast write queues used `std::vector::erase(begin())`, requiring O(N) shifts per write and degrading under stress-testing load.

✓ Resolved: Replaced with `std::deque` to achieve strict O(1) popped packet dispatches.